

Fiori Elements 開發課程 2：專案結構解讀

環境：BTP ABAP Environment Trial (沿用 fe01 建立的 `fe01_connection_test` 專案，本課未產生新物件)

Lecture

這一課不重新建專案，直接深度解析 fe01 已經產生的東西

fe01 把重點放在「怎麼連上系統、產生專案」，對產生出來的檔案只給了一句話帶過 (`manifest.json` / `ui5.yaml` / `package.json` 各自的角色)。這一課要做的是逐項拆開 `fe01_connection_test` / 這個資料夾，搞懂每個檔案實際存了什麼、為什麼長這樣——不需要再跑一次 Generator，直接讀已經有的專案就好。

下面每一段引用的內容，都是直接讀取 `fe01_connection_test` / 裡的真實檔案得到的 (不是照抄官方文件的通用範例)，所以你可以自己打開對照。

`manifest.json`：App 的「總設定檔」，逐段解讀

`sap.app`——App 的身分與資料來源：

```
"sap.app": {
  "id": "fe01connectiontest",
  "dataSources": {
    "annotation": { "type": "ODataAnnotation", "uri": "annotations/annotation.xml" },
    "mainService": {
      "uri": "/sap/opu/odata4/sap/zrc08_sb/srvd/sap/zrc08_sd/0001/",
      "type": "OData",
      "settings": { "annotations": ["annotation"], "odataVersion": "4.0" }
    }
  }
}
```

`dataSources` 裡有兩個資料來源：`mainService` 是真正的 OData 服務 (fe01 已經核對過)，`annotation` 是本機額外的 **UI Annotation** 檔案 (`annotations/annotation.xml`)——注意 `mainService.settings.annotations` 陣列裡引用了它，代表這個 App 實際生效的 UI 標記 = 「後端 `$metadata` 自帶的 Annotation」+ 「這個本機檔案疊加的 Annotation」兩者合併。

`sap.ui5.routing`——List Report / Object Page 怎麼串起來：

```
"routing": {
  "routes": [
    { "pattern": "?:?query:", "name": "NoteList", "target": "NoteList" },
    { "pattern": "Note({key}):?query:", "name": "NoteObjectPage", "target": "NoteObjectPage" }
  ],
  "targets": {
    "NoteList": {
      "type": "Component", "name": "sap.fe.templates.ListReport",
      "options": { "settings": {
        "contextPath": "/Note",
        "navigation": { "Note": { "detail": { "route": "NoteObjectPage" } } } },
      "controlConfiguration": {
        "@com.sap.vocabularies.UI.v1.LineItem": { "tableSettings": { "type": "ResponsiveTable"
```

```

} }
    }
  } }
},
"NoteObjectPage": {
  "type": "Component", "name": "sap.fe.templates.ObjectPage",
  "options": { "settings": { "editableHeaderContent": false, "contextPath": "/Note" } }
}
}
}

```

幾個關鍵對應關係：

- 這裡完全沒有你自己寫的 **View / Controller**——`target.name` 直接指到 SAP 官方框架元件 `sap.fe.templates.ListReport` / `sap.fe.templates.ObjectPage`，畫面完全是這兩個範本讀取 `contextPath` 指到的 CDS Entity (`/Note`) + Annotation 動態生成的。這就是「Fiori Elements」這個名字的意思：你只給設定 (Annotation + 這份 routing 設定)，框架自己生出畫面，不是你手刻 XML View。
- **pattern** 是 **URL 路由規則**：`:?query:` 代表「可有可無的查詢參數」(List Report 首頁不需要任何路徑參數)；`Note({key}):?query:` 的 `{key}` 是**真正的 URL 參數**，對應點進某一筆 Note 時網址列會出現的 `Note(RC05TEST02,IsActiveEntity=true)` 這種技術 Key。
- **navigation.Note.detail.route** 就是「List Report 點一列資料，該跳去哪個 route」的設定——這裡指到 `NoteObjectPage`，兩個 Target 因此被串在一起。
- **controlConfiguration** 裡的 **tableSettings.type: "ResponsiveTable"**——這正是 fe01 Generator 精靈第 10 步你選的 **Table Type: Responsive** 落地的地方，證實 Generator 那一步問的問題確實對應到這裡的設定。
- **editableHeaderContent: false**——Object Page 標題列的欄位預設不能直接編輯，要先按 **Edit** 按鈕才能改 (Draft 進入編輯模式)，這是官方範本針對 Draft 實體的標準行為。

sap.fe——這是全域的 Fiori Elements 執行期設定 (跟 `sap.ui5.routing.targets.*.options.settings` 這種「單一頁面設定」不同層級)：

```
"sap.fe": { "app": { "enableLazyLoading": true } }
```

這一課只有這一個設定 (延遲載入子元件，優化效能)，之後幾課如果用到 **sap.fe** 層級的其他開關，會在對應課程說明。

webapp/ 資料夾逐項介紹：哪些是真檔案，哪些是「工具動態生成」的虛擬資源

fe01 已經教過一個重要觀念：**Application Info** 面板不是檔案，是擴充套件動態渲染的畫面。**這個觀念在 webapp/ 資料夾裡還有另一個例子**，一併在這裡講清楚：

檔案 / 路徑	是真檔案嗎	說明
<code>Component.js</code>	✅ 真檔案	整個 App 的進入點，內容只有 5 行： <code>Component.extend("fe01connectiontest.Component", { metadata: { manifest: "json" } })</code> ——繼承 <code>sap/fe/core/AppComponent</code> ，把所有設定都委派給 <code>manifest.json</code> ，本身幾乎沒有邏輯
<code>index.html</code>	✅ 真檔案	standalone 模式的進入點 (<code>npm run start-noflp</code> 用的就是這個)，直接載入 UI5 Bootstrap + 掛載 Component，沒有 Fiori Launchpad 外殼
<code>test/flp.html</code> (瀏覽器網址列打的那個)	⚠️ 不是真檔案	<code>find</code> 這個資料夾找不到 <code>flp.html</code> ，但瀏覽器打得開——它是 <code>ui5.yaml</code> 裡 <code>fiori-tools-preview</code> middleware 執行期動態生成的「Local FLP Sandbox」(<code>npm start</code> 的 log 也印

檔案 / 路徑	是真檔案嗎	說明
		過 <code>Using sandbox template for UI5 version: ...</code> 這一行) · 模擬正式環境 Fiori Launchpad 的外殼 · 讓你在本機也能用跟正式環境一樣的方式 (Tile 點進去) 預覽 App · 不用另外架一個真正的 Launchpad
<code>i18n/i18n.properties</code>	✓ 真檔案	文字資源檔 · 目前只有 <code>appTitle</code> / <code>appDescription</code> 兩個 key (對應 <code>manifest.json</code> 裡的 <code>{{appTitle}}</code> / <code>{{appDescription}}</code> 佔位符)
<code>annotations/annotation.xml</code>	✓ 真檔案 · 但目前是空殼	只有 17 行 · <code><Schema Namespace="local"></code> 底下完全沒有任何 <code><Annotations></code> 內容——因為這一課的 <code>@UI.*</code> 標記 (<code>HeaderInfo</code> / <code>Facets</code> / <code>LineItem</code> / <code>Identification</code>) 全部已經在後端 CDS Metadata Extension 裡寫好 · 隨 <code>\$metadata</code> 一起送過來 · 這個本機檔案不用重複定義。它的真正用途是前端想要疊加或覆寫某個 Annotation 時 · 不用改 ABAP · 直接改這個檔案就好 (下面動手練習會示範)
<code>localService/mainService/metadata.xml</code>	✓ 真檔案 (531 行)	Generator 精靈連線時抓回來的 <code>\$metadata</code> 本機副本——用 <code>grep</code> 可以在這個檔案裡直接找到 <code>SAP__UI.HeaderInfo</code> / <code>SAP__UI.Facets</code> / <code>SAP__UI.LineItem</code> / <code>SAP__common.DraftRoot</code> 這些標記的完整內容 · 證實 rc08 在 ABAP CDS 端寫的 Metadata Extension · 最終就是編譯成這裡看到的樣子。⚠ 這份副本主要給 Mock Server / 型別提示用 · 執行期實際顯示畫面時 · App 打的是真正的後端 <code>\$metadata</code> · 不是這份本機副本——如果之後 ABAP 端的 Annotation 改了 · 這份本機檔案要重新用 Generator (或對應指令) 刷新 · 不會自動同步
<code>localService/mainService/srvd_f4/...</code>	✓ 真檔案	Value Help 的 Metadata (例如 <code>I_DraftAdministrativeUserVH</code> · Draft 建立人 / 異動人的 Value Help) · 這是 fe01 第 9 步「Download value help metadata」選 Yes 才會抓下來的東西——選 No 的話這個子資料夾就不會存在
<code>test/integration/*.gen.js</code>	✓ 真檔案 (Generator 自動產生)	<code>NoteListJourney.gen.js</code> / <code>NoteObjectPageJourney.gen.js</code> 是 OPA5 (One Page Acceptance · SAPUI5 官方整合測試框架) 的測試骨架 · 根據 <code>manifest.json</code> 的 routing 設定自動生成——檔名對應到 List Report / Object Page 兩個頁面 · <code>.gen.js</code> 副檔名代表「工具生成、之後重跑 Generator 可能會被覆蓋」 · 這門課不深入 (整合測試不是這門課的主題) · 但值得知道它的存在

三份 `ui5*.yaml` 設定檔的差異

專案裡其實有三份 UI5 Tooling 設定檔 · `npm start` / `npm run start-local` / `npm run start-mock` 分別對應：

檔案	對應指令	連線對象	用途
<code>ui5.yaml</code>	<code>npm start</code>	真實後端 (<code>fiori-tools-proxy</code>) + 遠端 UI5 Runtime (<code>ui5.sap.com</code>)	fe01 用的就是這個——最貼近正式環境的行為 · 但需要網路連線 + 登入

檔案	對應指令	連線對象	用途
ui5-mock.yaml	npm run start-mock	完全不連後端，改用 sap-fe-mockserver middleware 根據 metadata.xml 自動生成假資料	適合離線開發、或後端還沒 Publish 時先開發畫面邏輯
ui5-local.yaml	npm run start-local	同時設定了 sap-fe-mockserver + 真實後端 fiori-tools-proxy，且多了一段 framework: 區塊指定把 SAPUI5 函式庫真正下載到本機（不透過 Proxy 連 ui5.sap.com）	適合完全離線環境（沒有網路也能開發，UI5 函式庫本身也不用連線）

這三份設定檔都是 **Generator** 精靈自動產生的，不用自己手刻——這一課只需要知道它們的差異，之後真的要用到離線 / Mock 開發模式時，直接換一個指令即可。

⚠️⚠️ npm start 怎麼觸發 Proxy：跟一般前端開發（webpack-dev-server / Vite）的 Proxy 概念是同一件事

npm start 執行的 fiori run 底層是 **ui5 serve**——UI5 Tooling 官方的開發伺服器，跑在 Node.js 上，架構跟 Express / Connect 同一類：一條 **Middleware Chain**，每個請求依序經過設定好的一串 middleware 處理。ui5.yaml 的 server.customMiddleware 就是在宣告這條 Chain 裡要插入哪些 middleware：

```
server:
  customMiddleware:
    - name: fiori-tools-proxy
      afterMiddleware: compression
      configuration:
        ui5:
          path: [/resources, /test-resources]
          url: https://ui5.sap.com
        backend:
          - path: /sap
            url: https://xxxxx.abap.ap21.hana.ondemand.com
            authenticationType: reentranceTicket
```

afterMiddleware: compression 指定 fiori-tools-proxy 插在內建的 compression middleware 之後，跟 Express 裡 app.use(someMiddleware) 的宣告方式是同一個概念，只是這裡用 YAML 宣告順序，不是寫程式碼手動 .use()。

實際請求流程：

1. 瀏覽器打開 localhost:8080，頁面裡的 App 發出 AJAX 請求，例如 GET /sap/opu/odata4/sap/zrc08_sb/.../Note——因為 manifest.json 的 mainService.uri 是相對路徑，這個請求的目標還是 localhost:8080，瀏覽器不會碰到 CORS 問題（永遠打同一個 Origin）
2. Node.js 開發伺服器收到請求，Middleware Chain 依序檢查：fiori-tools-proxy 看到路徑符合 /sap 規則，攔截這個請求，在伺服器端（不受瀏覽器 CORS 限制）自己發一個新的 HTTP 請求，轉發到真正的後端
3. 因為設定了 authenticationType: reentranceTicket，這個 middleware 還會自動處理 OAuth 認證——沒有有效 Token 時觸發瀏覽器登入視窗，拿到後快取，之後每次轉發自動帶上
4. 後端回應串流回瀏覽器，瀏覽器完全不知道背後實際打的是哪個網域

這就是一般前端開發熟悉的 **Dev Server Proxy** 概念——跟 webpack-dev-server 的 devServer.proxy、Vite 的 server.proxy、Create React App package.json 裡的 "proxy"、Angular CLI 的 proxy.conf.json 解決的是同一個問題：本機開發伺服器要跟另一個 Origin 的真實後端溝通，讓瀏覽器永遠只跟同一個 Origin 講話，實際跨網域轉發交給 Node.js 伺服器進程處理（伺服器對伺服器沒有 CORS 限制）。**SAP Fiori tools** 額外加的值：一般 webpack/Vite proxy 通常只做「轉發」，認證要自己處理；fiori-tools-proxy 把 OAuth Reentrance Ticket（BTP）/ Basic Auth（地端）這整套認證流程也包進 proxy 自動處理，不用自己管理 Token。

ui5 那一段 (`path: [/resources, /test-resources] → https://ui5.sap.com`) 走的是同一套機制，只是轉發對象不同——本機專案沒有把完整的 SAPUI5 框架函式庫存在本地，這段規則把載入框架程式碼的請求轉發到 SAP 官方 CDN (除非用 `ui5-local.yaml` 那種指定 `framework`：把函式庫真正下載到本機的設定)。

`package.json` 的 `scripts`：每個指令在做什麼

Script	實際指令	用途
<code>start</code>	<code>fiori run --open "test/flp.html#app-preview"</code>	fe01 用的標準本機開發模式
<code>start-local</code>	帶 <code>ui5-local.yaml</code>	見上表，離線開發
<code>start-mock</code>	帶 <code>ui5-mock.yaml</code>	見上表，Mock 資料
<code>start-noflp</code>	開 <code>/index.html</code> (不透過 FLP Sandbox)	直接用 <code>index.html</code> 這個真檔案開啟，不經過虛擬的 <code>flp.html</code>
<code>build</code>	<code>ui5 build --config=ui5.yaml --clean-dest --dest dist</code>	打包成正式的靜態資源， <code>dist/</code> 資料夾 (<code>.gitignore</code> 已排除)——fe06 部署會用到
<code>deploy</code>	<code>fiori verify</code>	部署 (fe06 詳細教)
<code>deploy-config</code>	<code>fiori add deploy-config</code>	產生部署設定 (fe06 詳細教)
<code>lint</code>	<code>eslint ./</code>	程式碼風格檢查
<code>int-test</code>	跑 OPA5 整合測試 (<code>ui5-mock.yaml</code> + <code>opaTests.qunit.html</code>)	對應上面 <code>test/integration/*.gen.js</code> 那些測試骨架

學習目標

- 能講出 `manifest.json` 的 `sap.app` / `sap.ui5.routing` / `sap.fe` 三個區塊各自負責什麼，尤其是 `routing.routes` / `routing.targets` 怎麼把 List Report 跟 Object Page 串成一個可以互相導覽的 App
- 知道 Fiori Elements App 沒有手寫的 View / Controller——畫面是 `sap.fe.templates.ListReport` / `sap.fe.templates.ObjectPage` 這兩個官方範本元件，讀取 `contextPath` + Annotation 動態生成的
- 能區分 `webapp/` 底下「真檔案」跟「工具執行期動態生成、實際不存在」的資源 (`test/flp.html` 是後者，跟 fe01 教過的 `Application Info` 面板是同一類概念)
- 知道 `@UI.*` Annotation 目前全部來自後端 `metadata.xml` (能用 `grep` 在裡面找到 `HeaderInfo` / `Facets` / `LineItem` / `DraftRoot`)，`annotations/annotation.xml` 這個本機檔案目前是空的，但是前端不改 **ABAP**、疊加/覆寫 **Annotation** 的正式管道
- 能講出 `ui5.yaml` / `ui5-local.yaml` / `ui5-mock.yaml` 三者的差異與對應的 `npm run` 指令
- 能講出 `npm start` 怎麼觸發 `fiori-tools-proxy`：UI5 Tooling 的 Middleware Chain 架構 (跟 Express/Connect 同一類)、實際請求轉發流程，並能對照到一般前端開發熟悉的 `webpack-dev-server` / `Vite Proxy` 概念——知道兩者解決的是同一個問題 (避免 CORS，讓瀏覽器永遠打同一個 Origin)，差異在 Fiori tools 的 Proxy 多包了 OAuth / Basic Auth 認證自動化
- 知道 `Download value help metadata` 這個 Generator 選項具體會多產生 `localService/mainService/srvd_f4/` 這個子資料夾

物件清單

這一課不產生任何新的前端專案或 ABAP 物件——完全是對 fe01 已經產生的 `fe01_connection_test/` 做深度解析：

檔案	這一課的重點
<code>fe01_connection_test/webapp/manifest.json</code>	<code>routing.routes /</code> <code>routing.targets / sap.fe</code>
<code>fe01_connection_test/webapp/Component.js</code>	App 進入點
<code>fe01_connection_test/webapp/annotations/annotation.xml</code>	本機 Annotation 疊加機制 (動手練習會實際改這個檔案)
<code>fe01_connection_test/webapp/localService/mainService/metadata.xml</code>	後端 <code>\$metadata</code> 本機副本，含完整 <code>@UI.*</code> 標記
<code>fe01_connection_test/ui5.yaml / ui5-local.yaml / ui5-mock.yaml</code>	三種本機開發模式

動手練習

練習 1 (livereload 體驗)：`fe01_connection_test` 的 `npm start` 應該還在背景跑著 (`fiori-tools-appreload` `middleware`，`port 35729`)。打開 `webapp/i18n/i18n.properties`，把 `appTitle=FE01` 連線測試 改成別的文字，存檔——不用重新執行 `npm start`，瀏覽器分頁標題應該會自動更新。這證實 `livereload` 真的在運作，也讓你直接體驗到「改設定檔、瀏覽器自動反映」這個開發迴圈。

練習 2 (本機 Annotation 疊加，選做，稍有難度)：`webapp/annotations/annotation.xml` 目前是空殼。試著在 `<Schema Namespace="local">` 裡加一段 `<Annotations Target="...">`，針對 `Note` Entity 的某個欄位覆寫顯示行為 (例如把某個欄位在 `List Report` 表格裡隱藏，或改變欄位順序)——不用改任何 **ABAP** 物件，存檔後 `livereload` 應該會反映在畫面上。如果卡住，可以先跟我核對要寫的 XML 內容。

驗證方式

這一課沒有 **ABAP** 物件變更，驗證方式是「讀懂 + 能對應到真實檔案內容」：

1. 你能指出 `manifest.json` 裡哪一段決定了「點 `List Report` 一列會跳到 `Object Page`」
2. 練習 1：`livereload` 實測成功，瀏覽器標題文字即時反映 `i18n.properties` 的修改
3. (選做) 練習 2：`annotation.xml` 加了本機覆寫後，畫面行為確實改變，且沒有動到任何 **ABAP** 物件

思考題

1. `manifest.json` 的 `dataSources.mainService.settings.annotations` 是一個陣列 (`["annotation"]`)，如果之後這個 App 需要疊加第二份本機 Annotation 檔案 (例如區分「這是這個 App 專屬的客製化」跟「這是共用的擴充」)，你覺得要怎麼設定？ (提示：陣列可以放多個 ID，每個 ID 對應 `dataSources` 底下另一個 `ODataAnnotation` 型別的項目)
2. `localService/mainService/metadata.xml` 是本機的 `$metadata` 副本，執行期實際顯示畫面時卻是打真正的後端——如果 **ABAP** 端後來又加了一個欄位的 `@UI.lineItem`，這個本機副本沒有跟著更新，畫面上會不會反映最新的後端 Annotation？這個副本存在的意義究竟是什麼 (提示：想一想 `ui5-mock.yaml` 的 `Mock Server` 是根據誰產生假資料的)
3. `routing.targets.NoteObjectPage` 沒有設定 `navigation` (跟 `NoteList` 不同，`NoteList` 有一段 `navigation.Note.detail.route`)——這代表 `Object Page` 沒辦法再往下導覽了嗎？如果之後 (fe08) 要做 `Header-Item` 的 `Composition App`，你覺得子項目的導覽設定應該加在哪裡？

答案

見 `fe01_connection_test/webapp/manifest.json` (`routing` 段落)、
`fe01_connection_test/webapp/annotations/annotation.xml` (動手練習 2 完成後會有內容)、
`fe01_connection_test/webapp/localService/mainService/metadata.xml` (後端 Annotation 的完整內容，用 `grep "SAP__UI\"` 可以快速定位)。這一課沒有產生新的物件，答案就是對照著讀 `fe01` 已經建立的專案。